how to create a rag system, give me references

A RAG system combines retrieval over your own data with an LLM so answers are grounded in relevant external context instead of relying only on model pretraining. A solid first version is: ingest documents, chunk them, embed the chunks, store them in a vector database, retrieve top matches for each question, and pass those results to the model with a prompt that says to answer only from the retrieved context and admit when the answer is unknown. [1] [2] [3]

Core architecture

A typical RAG pipeline has five stages: loading data, indexing it, storing the index, querying it, and evaluating output quality. In practice, that means you turn files, PDFs, web pages, database rows, or API output into documents, split them into smaller chunks, convert those chunks into embeddings, store them in a vector store, and retrieve the most relevant chunks at query time.[2] [1]

Minimal build plan

1. Choose a data source, for example internal docs, manuals, support tickets, or product specs.[1]
2. Parse and clean the text, then split it into chunks; LangChain's example uses a recursive text splitter with chunking and overlap because large documents are harder to search and may not fit well in model context windows.[2]
3. Generate embeddings and store them in a vector store such as FAISS, Chroma, Pinecone, Qdrant, PGVector, or an in-memory store for prototypes.[2]
4. At runtime, retrieve the top relevant chunks for the user query and inject them into the prompt so the model answers from that context.[1] [2]
5. Evaluate groundedness, relevance, faithfulness, and latency, since both retrieval quality and response quality matter in RAG systems.[3] [1]

Design choices

There are two common patterns: an agentic RAG flow and a two-step RAG chain. Agentic RAG lets the model decide when and how to search, which is more flexible and can support multi-step retrieval, while a two-step chain always retrieves first and then makes a single generation call, which is often faster and easier to control.[2]

A few practical defaults:

- Start with chunk sizes around 500 to 1,000 characters or a few hundred tokens, with overlap, then tune empirically because chunking strongly affects retrieval quality.[2]
- Use hybrid retrieval later if needed, combining semantic search with keyword search and reranking, since this often improves relevance over plain vector search alone.[3]
- Add metadata filters early, such as document type, date, product, or team, because they reduce noise and improve retrieval precision.[1]

Security and quality

Your prompt should explicitly tell the model to treat retrieved text as data only and ignore instructions inside that text, because indirect prompt injection is a real risk in RAG pipelines. You should also return citations or source snippets in the UI, and test failure cases where retrieval finds irrelevant context or no context at all, because grounded but off-topic answers are still bad answers.[3] [1] [2]

References

Here are strong references to learn from:

- Google Cloud, "What is Retrieval-Augmented Generation (RAG)?" — clear conceptual overview of grounding, hybrid search, reranking, and evaluation: cloud.google.com/use-cases/retrieval-augmented-generation[3]
- LangChain Docs, "Build a RAG agent with LangChain" — practical step-by-step implementation with loading, splitting, vector storage, agentic RAG, two-step chains, and prompt-injection mitigation: docs.langchain.com/oss/python/langchain/rag[2]
- LlamaIndex Docs, "Introduction to RAG / Understanding RAG" — strong framework-level explanation of loading, indexing, storing, querying, retrievers, routers, postprocessors, and response synthesizers: developers.llamaindex.ai/python/framework/understanding/rag/[1]
- AWS, "What is RAG?" — concise enterprise-oriented overview and use cases: aws.amazon.com/what-is/retrieval-augmented-generation/[1]
- NVIDIA, "What Is Retrieval-Augmented Generation" — good overview of why RAG improves factuality and reliability: blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/[4]

Example mental model: think of RAG as "search first, answer second." The search step finds the right evidence; the generation step turns that evidence into a usable answer.[3] [1]

Would you like a simple Python starter template for RAG with LangChain or LlamaIndex?

⁂

1. https://aws.amazon.com/what-is/retrieval-augmented-generation/
2. https://en.wikipedia.org/wiki/Retrieval-augmented_generation
3. https://cloud.google.com/use-cases/retrieval-augmented-generation

4. https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/
5. https://medium.com/@imabhi1216/basic-to-advanced-rag-using-llamaindex-1-9270479df94f
6. https://www.databricks.com/blog/what-is-retrieval-augmented-generation
7. https://www.youtube.com/watch?v=o126p1QN_RI
8. https://www.llamaindex.ai/blog/a-cheat-sheet-and-some-recipes-for-building-advanced-rag-803a9d94c41b
9. https://wandb.ai/cosmo3769/RAG/reports/A-Gentle-Introduction-to-Retrieval-Augmented-Generation-RAG---Vmlldzo1MjM4Mjk1
10. https://medium.com/@o39joey/introduction-to-rag-with-python-langchain-62beeb5719ad
11. https://www.ibm.com/think/topics/retrieval-augmented-generation
12. https://docs.langchain.com/oss/python/langchain/rag
13. https://developers.llamaindex.ai/python/framework/understanding/rag/
14. https://www.weka.io/learn/guide/ai-ml/retrieval-augmented-generation/
15. https://www.youtube.com/watch?v=sVcwVQRHIc8